

Appendix C

Source Code

```
1 #pragma once
2
3 /***** Blynk IoT *****/
4 #define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID"
5 #define BLYNK_TEMPLATE_NAME "Self Watering Plant Template"
6 #define BLYNK_AUTH_TOKEN "<Redacted>"
7
8 /***** WiFi *****/
9 #define WIFI_SSID "*****"
10 #define WIFI_PASS "*****"
11
12
13 /***** Blynk Virtual Pins *****/
14 static const int VPIN_SOIL_PERCENT = 0; // V0
15 static const int VPIN_TANK_OK = 1; // V1
16 static const int VPIN_PUMP_STATE = 2; // V2
17 static const int VPIN_SETPPOINT = 10; // V10
18 static const int VPIN_MANUAL_WATER = 11; // V11
19
20 #include <Arduino.h>
21
22 /***** PINS *****/
23 static const int SOIL_ADC_PIN = 34; // ADC1
24 static const int RELAY_PIN = 25; // relay input pin
25 static const bool RELAY_ACTIVE_HIGH = true;
26 static const int THRESH_POT_PIN = 35;
27
28 /***** FORCE WATER BUTTON (physical) *****/
29 static const int FORCE_BTN_PIN = 13;
30 static const bool FORCE_BTN_ACTIVE_HIGH = false;
31
32 /***** WATER LEVEL SENSOR *****/
33 static const int WATER_LEVEL_PIN = 27;
34 // Your sensor: "low water is HIGH"
35 static const bool LOW_WATER_IS_HIGH = true;
36
37 /***** LOW WATER LED *****/
38 static const int LOW_WATER_LED_PIN = 14;
```

```

39 static const bool LED_ACTIVE_HIGH = true;
40
41 /***** BUTTON DEBOUNCE *****/
42 static const uint32_t BTN_DEBOUNCE_MS = 30;
43
44 /***** SERIAL PRINT *****/
45 static const uint32_t PRINT_PERIOD_MS = 500;
46
47 /***** SOIL MOISTURE CALIBRATION *****/
48 // Voltage increases with moisture
49 static const float SOIL_V_DRY = 0.80f;    // ~0% moisture
50 static const float SOIL_V_WET = 2.00f;    // ~100% moisture
51
52 /***** APP SETPOINT *****/
53 static const int DEFAULT_SETPOINT_PERCENT = 45; // target
54           moisture %
54 static const int LARGE_DELTA_PERCENT      = 10;
55 static const int HYSTERESIS_PERCENT      = 3;
56
57 /***** WATERING TIME *****/
58 static const uint32_t WATER_SMALL_MS = 1000;
59 static const uint32_t WATER_LARGE_MS = 1000;
60 static const uint32_t SOAK_DELAY_MS  = 9000;
61
62 /***** LCD (I2C 16x2) *****/
63 static const int I2C_SDA_PIN = 21;
64 static const int I2C_SCL_PIN = 22;
65 static const uint8_t LCD_ADDR = 0x3F; // common: 0x27 or 0x3F
66 static const uint32_t LCD_UPDATE_MS = 250;

```

Listing C.1: Configuration header (config.h)

```

1 #include "config.h"
2 #include <Arduino.h>
3 #include <WiFi.h>
4 #include <BlynkSimpleEsp32.h>
5
6 #include "relay.h"
7 #include "soil_sensor.h"
8 #include "watering_controller.h"
9 #include "water_level.h"
10 #include "indicator_led.h"
11 #include <Wire.h>
12 #include <LiquidCrystal_I2C.h>
13
14 LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
15
16 // ===== Objects =====
17 Relay relay;
18 SoilSensor soil;
19 WateringController ctrl;
20 WaterLevel waterLevel;
21 IndicatorLED lowWaterLED;
22
23 BlynkTimer timer;
24

```

```

25 // ===== Force Button debounce =====
26 static int lastBtnStable = HIGH;
27 static int lastBtnRead   = HIGH;
28 static uint32_t lastBtnChangeMs = 0;
29
30 // ===== Soil smoothing (EMA) =====
31 static float soilV_filt = NAN;
32
33 // ===== Setpoint from App (V10) =====
34 static int setpointPercent = DEFAULT_SETPOINT_PERCENT;
35 static int lastPotPercent = -1;
36 static const uint32_t POT_UPDATE_MS = 200;
37
38 // ----- Helpers -----
39 static float clampf(float x, float lo, float hi) { return (x < lo)
    ? lo : (x > hi ? hi : x); }
40 static int clampi(int x, int lo, int hi) { return (x < lo) ? lo :
    (x > hi ? hi : x); }
41
42 static int readThresholdPotPercent() {
43     int raw = analogRead(THRESH_POT_PIN); // 0..4095
44     int pct = (raw * 100) / 4095;         // 0..100
45     return clampi(pct, 0, 100);
46 }
47
48 static float percentToVoltage(int percent) {
49     percent = clampi(percent, 0, 100);
50     float v = SOIL_V_DRY + (SOIL_V_WET - SOIL_V_DRY) * (percent /
        100.0f);
51     float vmin = (SOIL_V_DRY < SOIL_V_WET) ? SOIL_V_DRY : SOIL_V_WET
        ;
52     float vmax = (SOIL_V_DRY > SOIL_V_WET) ? SOIL_V_DRY : SOIL_V_WET
        ;
53     return clampf(v, vmin, vmax);
54 }
55
56 static int voltageToPercent(float v) {
57     float vmin = (SOIL_V_DRY < SOIL_V_WET) ? SOIL_V_DRY : SOIL_V_WET
        ;
58     float vmax = (SOIL_V_DRY > SOIL_V_WET) ? SOIL_V_DRY : SOIL_V_WET
        ;
59     v = clampf(v, vmin, vmax);
60     float pct = (v - SOIL_V_DRY) / (SOIL_V_WET - SOIL_V_DRY) * 100.0
        f;
61     if (!isfinite(pct)) pct = 0;
62     return clampi((int)lroundf(pct), 0, 100);
63 }
64
65 static void applySetpointToController() {
66     int sp = clampi(setpointPercent, 0, 100);
67     int yPct = clampi(sp - LARGE_DELTA_PERCENT, 0, 100);
68
69     float Xv = percentToVoltage(sp);
70     float Yv = percentToVoltage(yPct);
71
72     float Xv_hys = percentToVoltage(clampi(sp - HYSTERESIS_PERCENT,

```

```

    0, 100));
73 float X = max(Xv_hys, Xv);
74 float Y = min(Yv, X - 0.01f);
75
76 WateringParams p;
77 p.X = X;
78 p.Y = Y;
79 p.smallMs = WATER_SMALL_MS;
80 p.largeMs = WATER_LARGE_MS;
81 p.soakMs = SOAK_DELAY_MS;
82 ctrl.setParams(p);
83
84 Serial.printf(">>> [SETPOINT] %d%% -> X=%.3fV, Y=%.3fV\n", sp, p
    .X, p.Y);
85 }
86
87 // ===== Handle physical force button
    =====
88 static void handleForceButton(bool tankOk) {
89     int raw = digitalRead(FORCE_BTN_PIN);
90     uint32_t now = millis();
91
92     if (raw != lastBtnRead) {
93         lastBtnRead = raw;
94         lastBtnChangeMs = now;
95     }
96
97     if ((now - lastBtnChangeMs) > BTN_DEBOUNCE_MS) {
98         if (lastBtnStable != lastBtnRead) {
99             lastBtnStable = lastBtnRead;
100
101             bool pressed = FORCE_BTN_ACTIVE_HIGH
102                 ? (lastBtnStable == HIGH)
103                 : (lastBtnStable == LOW);
104
105             if (pressed) {
106                 if (!tankOk) {
107                     Serial.println(">>> [BTN] Manual watering blocked: LOW
                        WATER");
108                 } else {
109                     Serial.println(">>> [BTN] Force SMALL watering");
110                     ctrl.requestManualSmallWater();
111                 }
112             }
113         }
114     }
115 }
116
117 // ===== Blynk callbacks =====
118
119 BLYNK_WRITE(VPIN_MANUAL_WATER) {
120     int v = param.asInt();
121     if (v == 1) {
122         bool tankOk = waterLevel.isOk();
123         if (!tankOk) {
124             Serial.println(">>> [APP] Manual watering blocked: LOW WATER

```

```

    ");
125     } else {
126         Serial.println(">>> [APP] Force SMALL watering");
127         ctrl.requestManualSmallWater();
128     }
129     Blynk.virtualWrite(VPIN_MANUAL_WATER, 0); // reset immediately
130 }
131 }
132
133 // ===== Periodic push to App =====
134 static void pushToBlynk() {
135     bool tankOk = waterLevel.isOk();
136     int soilPct = isfinite(soilV_filt) ? voltageToPercent(soilV_filt
137         ) : 0;
138
139     Blynk.virtualWrite(VPIN_SOIL_PERCENT, soilPct);
140     Blynk.virtualWrite(VPIN_TANK_OK, tankOk ? 1 : 0);
141     Blynk.virtualWrite(VPIN_PUMP_STATE, relay.isOn() ? 1 : 0);
142 }
143
144 static void pollThresholdPot() {
145     int potPct = readThresholdPotPercent();
146
147     // deadband to avoid jitter/spam
148     if (abs(potPct - lastPotPercent) < 1) return;
149
150     lastPotPercent = potPct;
151     setpointPercent = potPct;
152     applySetpointToController();
153
154     // OPTIONAL: if you still want to *display* it in the app (not
155     // control)
156     Blynk.virtualWrite(VPIN_SETPOINT, setpointPercent);
157 }
158
159 static void updateLCD() {
160     bool tankOk = waterLevel.isOk();
161     int soilPct = isfinite(soilV_filt) ? voltageToPercent(soilV_filt
162         ) : 0;
163
164     // Line 1: Moist and Setpoint
165     lcd.setCursor(0, 0);
166     lcd.print("M:");
167     if (soilPct < 100) lcd.print(' ');
168     if (soilPct < 10)  lcd.print(' ');
169     lcd.print(soilPct);
170     lcd.print("% T:");
171     if (setpointPercent < 100) lcd.print(' ');
172     if (setpointPercent < 10)  lcd.print(' ');
173     lcd.print(setpointPercent);
174     lcd.print("%");
175
176     // Line 2: Tank + Pump
177     lcd.setCursor(0, 1);
178     lcd.print("Tank:");

```

```

177     lcd.print(tankOk ? "OK " : "LOW");
178     lcd.print("Pump:");
179     lcd.print(relay.isOn() ? "ON " : "OFF");
180     lcd.print(" "); // clear tail
181 }
182
183 void setup() {
184     Serial.begin(115200);
185     Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
186     lcd.init();
187     lcd.backlight();
188     lcd.clear();
189     lcd.setCursor(0, 0);
190     lcd.print("Plant System");
191     lcd.setCursor(0, 1);
192     lcd.print("Booting...");
193     delay(200);
194
195     analogReadResolution(12);
196     pinMode(THRESH_POT_PIN, INPUT);
197
198     relay.begin(RELAY_PIN, RELAY_ACTIVE_HIGH);
199     soil.begin(SOIL_ADC_PIN);
200
201     pinMode(FORCE_BTN_PIN, INPUT_PULLUP);
202
203     waterLevel.begin(WATER_LEVEL_PIN, LOW_WATER_IS_HIGH);
204     lowWaterLED.begin(LOW_WATER_LED_PIN, LED_ACTIVE_HIGH);
205
206     ctrl.begin(&relay);
207     applySetpointToController();
208
209     WiFi.mode(WIFI_STA);
210     WiFi.setSleep(false);
211
212     int n = WiFi.scanNetworks();
213     Serial.printf("Scan found %d networks\n", n);
214     for (int i = 0; i < n; i++) {
215         Serial.printf("%d: %s (RSSI %d)\n", i, WiFi.SSID(i).c_str(),
216             WiFi.RSSI(i));
217     }
218
219     WiFi.begin(WIFI_SSID, WIFI_PASS);
220
221     uint32_t t0 = millis();
222     while (WiFi.status() != WL_CONNECTED && millis() - t0 < 15000) {
223         delay(300);
224         Serial.print(".");
225     }
226     Serial.println();
227
228     Blynk.config(BLYNK_AUTH_TOKEN);
229     Blynk.connect(5000);
230
231     timer.setInterval(1000L, pushToBlynk);
232     timer.setInterval(LCD_UPDATE_MS, updateLCD);

```

```

232 timer.setInterval(POT_UPDATE_MS, pollThresholdPot);
233
234 Serial.println("=== Auto Watering System (Blynk + Physical/App
    Manual Water + Low Water Safety) ===");
235 }
236
237 void loop() {
238   Blynk.run();
239   timer.run();
240   if (WiFi.status() != WL_CONNECTED) {
241     WiFi.reconnect();
242   }
243   if (WiFi.status() == WL_CONNECTED && !Blynk.connected()) {
244     Blynk.connect(2000);
245   }
246
247   bool tankOk = waterLevel.isOk();
248
249   // --- Flash LED when tank is LOW ---
250   static uint32_t lastBlinkMs = 0;
251   static bool ledOn = false;
252
253   if (!tankOk) {
254     if (millis() - lastBlinkMs >= 500) { // 500ms toggle = 1Hz
255       blink
256       lastBlinkMs = millis();
257       ledOn = !ledOn;
258       lowWaterLED.set(ledOn);
259     }
260   } else {
261     ledOn = false;
262     lowWaterLED.set(false);
263   }
264
265   float v = soil.readVoltageAvg(1, 0);
266   if (!isfinite(soilV_filt)) soilV_filt = v;
267   soilV_filt = 0.9f * soilV_filt + 0.1f * v;
268
269   handleForceButton(tankOk);
270
271   ctrl.update(soilV_filt, tankOk);
272
273   static uint32_t lastPrint = 0;
274   uint32_t now = millis();
275   if (now - lastPrint >= PRINT_PERIOD_MS) {
276     lastPrint = now;
277     Serial.printf("[STAT] Soil=%.3fV (%d%%) | Tank=%s | Pump=%s |
        State=%s | ",
278       soilV_filt,
279       voltageToPercent(soilV_filt),
280       tankOk ? "OK" : "LOW",
281       relay.isOn() ? "ON" : "OFF",
282       ctrl.stateName());
283     Serial.printf("WiFi=%d IP=%s | Blynk=%s\n",
284       (int)WiFi.status(),

```

```

285         WiFi.isConnected() ? WiFi.localIP().toString().c_str
           () : "0.0.0.0",
286         Blynk.connected() ? "YES" : "NO");
287     }
288 }

```

Listing C.2: Main program (main.cpp)

```

1  #include "watering_controller.h"
2
3  void WateringController::begin(Relay* relay) {
4      _relay = relay;
5      _state = IDLE;
6      _manualReq = false;
7  }
8
9  void WateringController::setParams(const WateringParams& p) {
10     _p = p;
11 }
12
13 const char* WateringController::stateName() const {
14     switch (_state) {
15         case IDLE:      return "IDLE";
16         case WATERING:  return "WATERING";
17         case SOAK:      return "SOAK";
18         case BLOCKED:   return "BLOCKED";
19     }
20     return "UNK";
21 }
22
23 void WateringController::requestManualSmallWater() {
24     _manualReq = true;
25 }
26
27 void WateringController::forceStopAndBlock() {
28     if (_relay) _relay->off();
29     _manualReq = false;
30     _state = BLOCKED;
31 }
32
33 void WateringController::update(float V, bool tankOk) {
34     uint32_t now = millis();
35
36     // ===== Safety: tank low => block ALL watering (auto + manual)
37     // =====
38     if (!tankOk) {
39         if (_state != BLOCKED) {
40             Serial.println(">>> [SAFETY] Low water detected -> STOP &
              BLOCK watering");
41         }
42         forceStopAndBlock();
43         return;
44     } else {
45         // tank recovered: leave blocked state
46         if (_state == BLOCKED) {
47             Serial.println(">>> [SAFETY] Tank OK -> resume IDLE");

```

```

47     _state = IDLE;
48 }
49 }
50
51 // ----- Manual override: water once immediately (small) -----
52 if (_manualReq) {
53     _manualReq = false;
54
55     // If already watering, ignore to avoid conflicts
56     if (_state != WATERING) {
57         _thisWaterMs = _p.smallMs;
58
59         Serial.printf(">>> [MANUAL] Force watering SMALL (%lu ms)\n"
60             ,
61                 (unsigned long)_thisWaterMs);
62
63         _relay->on();
64         _tEnd = now + _thisWaterMs;
65         _state = WATERING;
66     }
67 }
68
69 switch (_state) {
70     case IDLE: {
71         if (V < _p.X) {
72             _thisWaterMs = (V < _p.Y) ? _p.largeMs : _p.smallMs;
73
74             Serial.printf(">>> [WATER] V=%.3f < X=%.2f | amount=%s (%
75                 lu ms)\n",
76                     V, _p.X, (V < _p.Y) ? "LARGE" : "SMALL",
77                     (unsigned long)_thisWaterMs);
78
79             _relay->on();
80             _tEnd = now + _thisWaterMs;
81             _state = WATERING;
82         }
83         break;
84     }
85
86     case WATERING: {
87         if ((int32_t)(now - _tEnd) >= 0) {
88             _relay->off();
89             _tEnd = now + _p.soakMs;
90             _state = SOAK;
91
92             Serial.printf(">>> [SOAK] %lu ms (ignore sensor)\n",
93                 (unsigned long)_p.soakMs);
94         }
95         break;
96     }
97
98     case SOAK: {
99         if ((int32_t)(now - _tEnd) >= 0) {
100             _state = IDLE;
101             Serial.println(">>> [SOAK] End. Re-check soil.");
102         }
103     }
104 }

```

```

101     break;
102 }
103
104     case BLOCKED:
105         break;
106 }
107 }

```

Listing C.3: Watering controller (watering_controller.cpp)

```

1  #pragma once
2  #include <Arduino.h>
3  #include "relay.h"
4
5  struct WateringParams {
6      float X;                // V < X => water (SMALL)
7      float Y;                // V < Y => water (LARGE), Y < X
8      uint32_t smallMs;       // small water time
9      uint32_t largeMs;       // large water time
10     uint32_t soakMs;         // soak delay after watering
11 };
12
13 class WateringController {
14 public:
15     void begin(Relay* relay);
16     void setParams(const WateringParams& p);
17
18     // Update controller with new soil voltage + tank OK flag (
19     // safety)
20     void update(float soilV, bool tankOk);
21
22     const char* stateName() const;
23
24     // Manual force watering request (one-shot)
25     void requestManualSmallWater();
26
27     // Safety: immediately stop pump and enter blocked state until
28     // tankOk is true
29     void forceStopAndBlock();
30
31 private:
32     enum State { IDLE, WATERING, SOAK, BLOCKED };
33
34     Relay* _relay = nullptr;
35     WateringParams _p {1.2f, 0.8f, 500, 1500, 12000};
36
37     State _state = IDLE;
38     uint32_t _tEnd = 0;
39     uint32_t _thisWaterMs = 0;
40
41     bool _manualReq = false;
42 };

```

Listing C.4: Watering controller header (watering_controller.h)

```

1  #pragma once

```

```

2  #include <Arduino.h>
3
4  class SoilSensor {
5  public:
6      void begin(int adcPin);
7      float readVoltageAvg(int samples, int sampleDelayMs) const;
8
9  private:
10     int _adcPin = -1;
11 };

```

Listing C.5: Soil sensor header (soil_sensor.h)

```

1  #include "soil_sensor.h"
2
3  void SoilSensor::begin(int adcPin) {
4      _adcPin = adcPin;
5      analogReadResolution(12);
6  }
7
8  float SoilSensor::readVoltageAvg(int samples, int sampleDelayMs)
9      const {
10     long sum_mv = 0;
11     for (int i = 0; i < samples; i++) {
12         sum_mv += analogReadMilliVolts(_adcPin); // mV
13         delay(sampleDelayMs);
14     }
15     return (sum_mv / (float)samples) / 1000.0f; // V

```

Listing C.6: Soil sensor module (soil_sensor.cpp)

```

1  #pragma once
2  #include <Arduino.h>
3
4  class Relay {
5  public:
6      void begin(int pin, bool activeHigh);
7      void on();
8      void off();
9      bool isOn() const;
10
11  private:
12     int _pin = -1;
13     bool _activeHigh = true;
14     bool _on = false;
15 };

```

Listing C.7: Relay header (relay.h)

```

1  #include "relay.h"
2
3  void Relay::begin(int pin, bool activeHigh) {
4      _pin = pin;
5      _activeHigh = activeHigh;
6      pinMode(_pin, OUTPUT);

```

```

7   off();
8 }
9
10 void Relay::on() {
11     _on = true;
12     digitalWrite(_pin, _activeHigh ? HIGH : LOW);
13 }
14
15 void Relay::off() {
16     _on = false;
17     digitalWrite(_pin, _activeHigh ? LOW : HIGH);
18 }
19
20 bool Relay::isOn() const {
21     return _on;
22 }

```

Listing C.8: Relay module (relay.cpp)

```

1  #pragma once
2  #include <Arduino.h>
3
4  class WaterLevel {
5  public:
6      void begin(int pin, bool lowIsHigh);
7      bool isLow() const;           // true = low water detected
8      bool isOk() const;           // true = sufficient water
9      int  raw() const;
10
11 private:
12     int  _pin = -1;
13     bool _lowIsHigh = true;
14 };

```

Listing C.9: Water level sensor header (water_level.h)

```

1  #include "water_level.h"
2
3  void WaterLevel::begin(int pin, bool lowIsHigh) {
4      _pin = pin;
5      _lowIsHigh = lowIsHigh;
6      // Many water level modules are open-collector / floating -> use
7      // pullup
8      pinMode(_pin, INPUT_PULLUP);
9  }
10
11 bool WaterLevel::isLow() const {
12     int v = digitalRead(_pin);
13     // if "low water is HIGH": HIGH means LOW-WATER
14     return _lowIsHigh ? (v == HIGH) : (v == LOW);
15 }
16
17 bool WaterLevel::isOk() const {
18     return !isLow();
19 }

```

```

20 int WaterLevel::raw() const {
21     return digitalRead(_pin);
22 }

```

Listing C.10: Water level sensor (water_level.cpp)

```

1  #pragma once
2  #include <Arduino.h>
3
4  class IndicatorLED {
5  public:
6      void begin(int pin, bool activeHigh);
7      void set(bool on);
8
9  private:
10     int _pin = -1;
11     bool _activeHigh = true;
12 };

```

Listing C.11: Indicator LED header (indicator_led.h)

```

1  #include "indicator_led.h"
2
3  void IndicatorLED::begin(int pin, bool activeHigh) {
4      _pin = pin;
5      _activeHigh = activeHigh;
6      pinMode(_pin, OUTPUT);
7      set(false);
8  }
9
10 void IndicatorLED::set(bool on) {
11     digitalWrite(_pin, (_activeHigh ? on : !on) ? HIGH : LOW);
12 }

```

Listing C.12: Indicator LED (indicator_led.cpp)